Architecting the Monadic Data Plane: High-Performance Integration of GPUDirect, DPDK gpudev, and BlueField SNAP for Zero-Host-CPU Emulation

---

Author: Ping Long, Chief Systems Architect, Lead Researcher, SiliconLanguage Foundry
***Contact**: LinkedIn | GitHub | plongpingl@gmail.com*

Abstract

The modern landscape of hyperscale infrastructure is undergoing a fundamental transformation as artificial intelligence (AI) workloads—particularly large language models (LLMs)—outpace the capabilities of traditional CPU-centric I/O architectures. In response, this research presents the Monadic Data Plane, a "Zero-Host-CPU" paradigm where data movement and transformation occur entirely within the peripheral domain. By orchestrating NVIDIA GPUDirect RDMA, the Data Plane Development Kit (DPDK) gpudev library, and NVIDIA BlueField SNAP, this architecture reduces the host CPU to a mere configuration agent.

To achieve a polymorphic, hardware-agnostic design, the architecture elevates its scale-out transport layer by adopting Libfabric (OFI), while utilizing NVIDIA's NVLink fabric for sub-microsecond, intra-node peer-to-peer scaling. Crucially, these asynchronous NVMe storage events and remote network transfers are unified under a novel C++23 "Monadic Sovereign Control Plane," which utilizes functional design patterns (such as std::expected and std::execution) for lock-free, predictive state management. Ultimately, this framework provides a highly deterministic, hardware-accurate blueprint capable of directly offloading massive KV caches and model activations to NVMe storage, effectively circumventing the GPU memory wall in next-generation scale-out AI infrastructure.

Evolution of High-Performance Data Movement and Market Landscape

The pursuit of zero-copy, kernel-bypass data movement is not a new endeavor, but its application in the unified GPU-DPU space represents the current frontier of systems engineering. To understand the state of the art, it is necessary to examine the existing frameworks that have attempted to bridge the gap between high-speed networking and heterogeneous compute.

Prior Art in Distributed Storage and AI Frameworks

DeepSeek 3FS, also known as the Fire-Flyer File System, serves as a primary example of a production-scale system designed for the extreme I/O requirements of AI training. [1] 3FS utilizes an RDMA-centric design that allows compute nodes to read from and write to remote NVMe storage without intermediate CPU copies or kernel involvement. [1] By employing 200 Gbps InfiniBand and achieving aggregate read bandwidths of 6.6 TiB/s, 3FS demonstrates the viability of saturating network links through asynchronous zero-copy APIs. [1] While current 3FS deployments do not explicitly implement GPU-direct data placement, the architecture identifies GPUDirect RDMA and Storage as the logical next steps for further reducing host mediation and PCIe traffic. [1]

The Distributed Asynchronous Object Storage (DAOS) framework represents another critical

component of this landscape. Recent efforts have demonstrated that offloading the DAOS client to an NVIDIA BlueField-3 DPU preserves RDMA efficiency while enabling DPU-resident features like multi-tenant isolation and inline services such as encryption. [1] This offloading demonstrates that the DPU can effectively manage the RDMA control plane, allowing the data plane to remain autonomous and high-performing. [1]

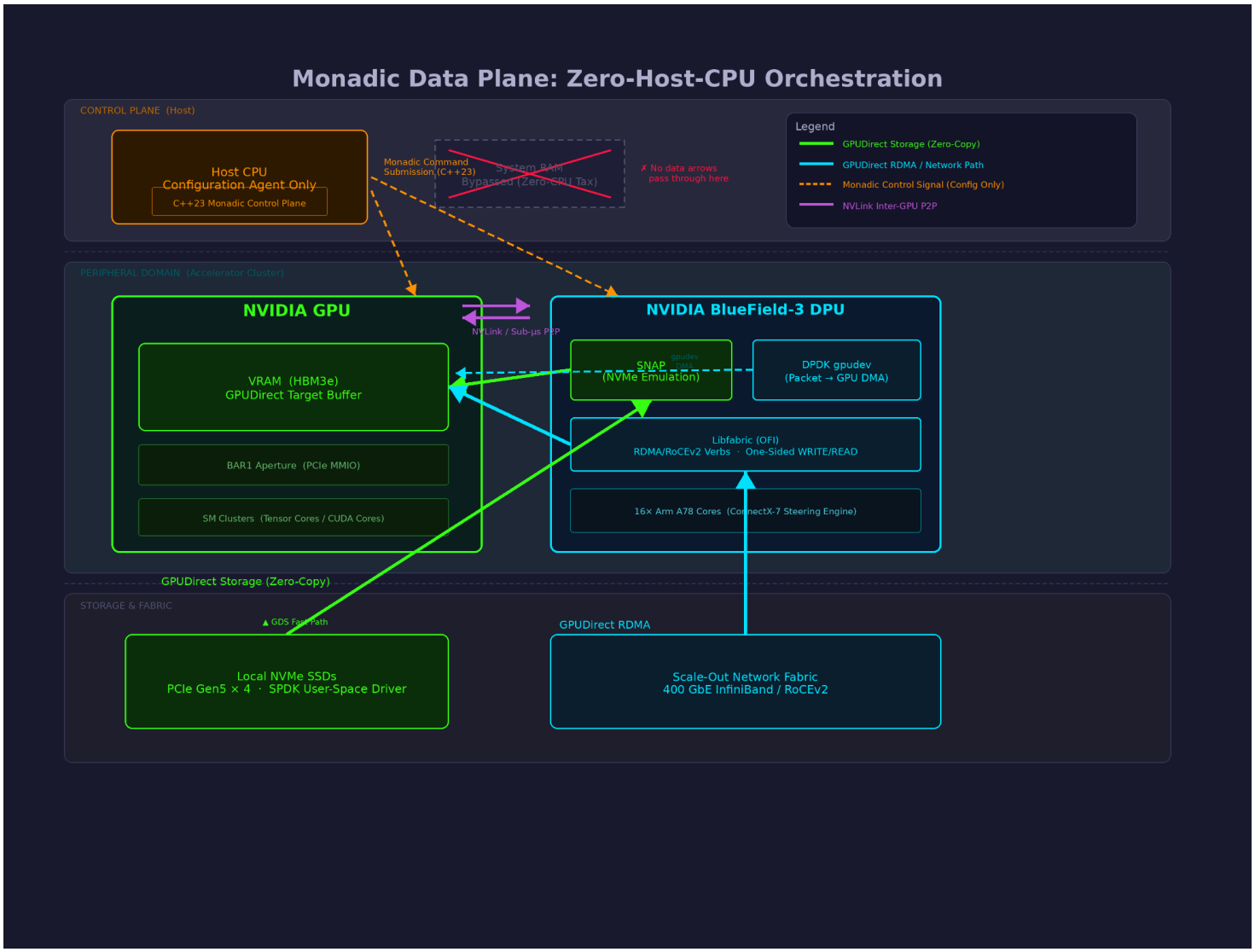
NVIDIA Magnum IO provides the commercial optimization stack for these technologies, offering a unified platform for multi-GPU and multi-node systems. [2] Magnum IO encompasses GPUDirect Storage (GDS), GPUDirect RDMA (GDR), and NVMe SNAP, providing the necessary drivers and libraries to bypass the CPU and system RAM during data transfers. [2, 3] The following table summarizes the key distinctions between these prior art systems and the proposed dataplane_emu architecture.

Feature	**DeepSeek 3FS**	**Intel DAOS (Offloaded)**	**NVIDIA Magnum IO**	**Proposed Monadic Data Plane**
Primary Data Path	RDMA to Host Mem	RDMA to Host/DPU Mem	GPU-Direct Storage/RDMA	SmartNIC to GPU VRAM
Control Path	CPU-driven io_uring	DPU-offloaded RDMA client	Library-based (cuFile)	C++ Monadic State Machine
Kernel Involvement	Bypass (RDMA)	Bypass (RDMA)	Bypass (GDS/GDR)	True Zero-Kernel (MMIO-only)
Storage Abstraction	Distributed FS	Object Store	Multi-layer SDK	Virtualized SNAP Block Dev
Primary Workflow	AI Training (3FS-client)	HPC / Object Storage	AI/HPC Acceleration	Hardware-Accurate Emulation

[1, 2, 4, 5]

The success of these systems highlights a critical trend: as GPU computing power increases, the demand for I/O bandwidth scales proportionally, necessitating a shift toward "In-Network Compute" where the DPU and GPU collaborate without host intervention. [2, 5]

Figure 1: High-Level Architecture of the Monadic Data Plane



Conceptual integration of the SmartNIC/DPU control plane with GPU-resident data buffers for kernel-bypass I/O.

The Memory Substrate: GPUDirect RDMA and BAR1 Mapping

The technical foundation of the Monadic Data Plane resides in the ability of the PCIe bus to support Peer-to-Peer (P2P) transactions between the SmartNIC and the GPU. This mechanism requires a deep understanding of the Base Address Register (BAR) mapping and the kernel-mode drivers that facilitate hardware visibility.

PCIe BAR1 and the GPU Aperture

NVIDIA GPUs utilize BAR1 as a memory-mapped I/O (MMIO) window that allows external devices—such as a BlueField DPU or another GPU—to directly access the GPU's video RAM (VRAM). [6] At system initialization, the BIOS or the kernel reserves a memory address range for each PCIe device's memory regions. [7] If a GPU's BAR1 space is not properly exposed or is insufficient in size, GPUDirect RDMA cannot establish a direct hardware path. [8, 9]

In modern data center GPUs, such as the NVIDIA A100 or H100, the BAR1 size can be configured to span the entire VRAM capacity, which is essential for hyperscale workloads. [9] However, on older or misconfigured systems, the BAR1 may be limited (e.g., 256 MiB), forcing the software stack into "Compatibility Mode". [8, 9, 10] In this mode, the cuFile library falls back to using pread/pwrite through host memory bounce buffers, which violates the zero-host-CPU requirement of our architecture. [8]

Kernel-Level Synchronization: nvidia-peermem and DMA-BUF

The orchestration of P2P DMA requires the participation of the Linux kernel to pin memory and establish address translation entries in the NIC. [7, 11] Traditionally, the nvidia-peermem kernel module provided this support specifically for Mellanox InfiniBand Host Channel Adapters (HCAs), allowing them to directly read and write to GPU VRAM. [11, 12] Modern implementations are transitioning toward the DMA-BUF framework, which is a standardized Linux kernel mechanism for sharing buffers across hardware drivers. [13, 14] Using DMA-BUF eliminates the need for legacy proprietary modules and allows frameworks like NCCL and DPDK to auto-detect hardware support for zero-copy transfers. [9, 14]

The registration process for a memory region to be used in GPUDirect RDMA involves four distinct steps: pinning the memory to prevent the OS from swapping it, storing address translation information in the NIC's internal translation lookaside buffer (TLB), setting access permissions, and generating local/remote keys (L_Key/R_Key) for the underlying transport, a process now abstracted by the Libfabric API. [7] In dataplane_emu, these steps must be abstracted into a unified C++ management class to ensure that memory is correctly prepared before the data plane initiates transfers.

DPDK gpudev and External Mempool Integration

The Data Plane Development Kit (DPDK) is the industry-standard framework for high-performance packet processing. The introduction of the gpudev library in DPDK 21.11 provides the necessary abstractions to treat the GPU as a first-class citizen in the networking stack. [15, 16]

GPU-Resident Mempools

To achieve zero-host-CPU data movement, the DPDK mempool—a continuous chunk of memory used for packet buffers (mbufs)—must reside in GPU VRAM. [16] The gpudev library allows the application to allocate this memory directly on the GPU and register it with the DPDK Environment Abstraction Layer (EAL). [15, 16]

The C++ integration for an external GPU mempool follows this sequence:

1. **Device Discovery:** Identify the GPU device using rte_gpu_is_valid and retrieve its properties. [15]
2. **Allocation:** Allocate a contiguous block of VRAM using rte_gpu_mem_alloc(gpu_id, size, alignment). [10, 16]
3. **Registration:** Register this external memory with the EAL using rte_extmem_register. This step is crucial for making the GPU memory addressable by DPDK's memory management logic. [17, 18]
4. **NIC Mapping:** Map the GPU memory into the NIC's address space using rte_dev_dma_map. This creates the hardware-level translation entries required for the NIC to perform DMA into the GPU. [16]

5. **Mempool Creation:** Instantiate the mempool using rte_pktmbuf_pool_create_extbuf, which associates the DPDK mbuf headers with the external GPU memory payloads. [16]

Communication Mechanisms and Control Flow

For the control flow, gpudev provides a "communication list" (rte_gpu_comm_list), which is a shared memory structure between the CPU and the GPU. [15, 16] This list contains an array of mbuf information, including data pointers and status flags such as RTE_GPU_COMM_LIST_READY or RTE_GPU_COMM_LIST_DONE. [16]

In dataplane_emu, a persistent CUDA kernel is launched during the configuration phase. This kernel resides on the GPU and polls the communication list in a tight loop. [16] When the NIC receives a packet and completes the DMA into the GPU mempool, the CPU (or a DPU-offloaded task) sets the status flag to READY. The persistent kernel, observing this state change, processes the packet directly in VRAM and then sets the flag to DONE to signal completion. [16] This polling mechanism avoids the latency associated with traditional interrupts and keeps the data path entirely within the GPU/NIC domain.

The mathematical model for polling efficiency in this context can be expressed by the hit-to-miss ratio of the progress engine. Let H be the number of times the progress engine finds a ready task and M be the number of idle polls. The efficiency η is given by:

$\eta = H / (H + M)$ [19]

High-performance emulators must maximize η by ensuring that packet bursts are sized to match the GPU's warp occupancy, minimizing idle cycles.

Elevating the Data Plane: Libfabric (OFI) as the Universal Transport Abstraction

While the integration of DPDK gpudev and GPUDirect provides the necessary low-level plumbing to achieve zero-host-CPU data movement, programming directly against raw RDMA Verbs or bare-metal DPDK APIs introduces significant hardware lock-in and steep application rewrite taxes. To achieve a truly polymorphic, "write-once, run-anywhere" architecture, the Monadic Data Plane elevates its networking dependency stack by adopting Libfabric (Open Fabrics Interfaces, or OFI) as its primary data plane engine. Libfabric serves as a highly optimized, transport-agnostic middleware layer residing between the application and the fabric hardware. By adopting Libfabric, the architecture explicitly separates the rigid, hardware-specific polling mechanisms from the high-level message routing logic, mirroring the exact hardware-software co-design utilized by exascale systems like Intel’s Distributed Asynchronous Object Storage (DAOS).

1. The Symbiosis of Libfabric and DPDK

A common misconception in user-space networking is that Libfabric replaces DPDK. In the Monadic Data Plane, they are strictly complementary, fulfilling distinct roles within the zero-kernel stack:

- **The Hardware Layer (DPDK gpudev):** DPDK is retained to manage the bare-metal hardware. It provides the Poll Mode Drivers (PMDs) and handles the physical memory pinning. Crucially, the gpudev library wraps cudaMalloc allocations, registering the GPU's Video RAM (VRAM) as an

external memory pool. Without DPDK, the network card would have no standardized mechanism to target GPU memory.

- **The Messaging Layer (Libfabric):** Operating directly above DPDK, Libfabric provides the "Monadic" programming interface. It manages connection state, handles completion events, and performs the complex semantic mapping required to translate application requests into the most efficient underlying hardware operations.

2. Hardware Independence via Provider Architecture

Libfabric achieves hardware independence through a dynamic "Provider" architecture. This allows the dataplane_emu C++ codebase to remain completely static while dynamically binding to the fastest available silicon at runtime.

- **NVIDIA AI Superclusters (Verbs Provider):** The engine utilizes the verbs provider to communicate directly with BlueField-3 DPUs [37] and ConnectX-7 NICs via RoCEv2 or InfiniBand. This provider translates OFI calls into low-level ibverbs for GPUDirect RDMA [39] efficiency.
- **AWS Graviton/Nitro Environments (EFA Provider):** On AWS infrastructure, the exact same application code seamlessly utilizes the EFA (Elastic Fabric Adapter) provider, achieving PCIe bypass via the Nitro system for scale-out LLM training.
- **HPC Supercomputers (CXI Provider):** In environments like the Aurora exascale supercomputer, the data plane utilizes the Slingshot (cxi) provider to interface with Cassini NICs and Rosetta switches, leveraging proprietary HPC extensions for small-message reductions.
- **Generic Cloud VMs (TCP/UDP Providers):** In the absence of specialized RDMA hardware, Libfabric seamlessly falls back to the tcp or udp socket providers, preserving monadic functionality and UCX [38] compatibility without code modification.

3. Industry Proof Points & Ecosystem Integration

Libfabric serves as the foundational transport for several exascale storage and AI collective communication projects relevant to the Monadic Data Plane:

- **NVIDIA NCCL & AWS EFA [41]:** Cloud providers use Libfabric plugins to map NVIDIA's connection-oriented NCCL APIs to connection-less reliable datagrams, enabling OS-bypass for massive LLM training on AWS instances.
- **Intel DAOS & oneCCL [42]:** Libfabric drives the Mercury RPC layer in exascale storage systems like DAOS and handles multi-node collective communications for Intel's deep learning frameworks across heterogeneous fabrics.
- **Advanced Transport & Storage [43]:** Next-generation AI infrastructure backends employ Libfabric for NUMA-aware rail selection and zero-copy intra-node GPU-to-GPU communication, aligning perfectly with our user-space messaging patterns.

4. Zero-Kernel Execution and CXL Disaggregation

Moving forward, the Monadic Data Plane must account for execution environments beyond x86, particularly **weakly-ordered silicon** such as ARM64 (e.g., NVIDIA Grace), which requires strict memory barrier considerations for lock-free SPSC queues [47]. To fully realize a "Zero-Kernel" architecture, the control plane can adopt Library OS concepts such as Microsoft's Demikernel [44] to unify RDMA and DPDK interfaces at the microsecond scale, while leveraging C++26 std::execution (P2300) [45] to handle asynchronous sender/receiver chains natively. Furthermore, embedding the

client stack on the DPU requires lightweight isolation; micro-kernel hypervisors like Alibaba Dragonball [46] offer the necessary multi-tenant security without the overhead of a full OS. Finally, the emergence of Compute Express Link (CXL) and PCIe device pooling [49, 50] will soon allow NVMe SSDs to be accessed directly as cache-coherent memory [48], entirely eliminating the need for block-level SPDK polling in future iterations of the data plane.

5. Implications for Large Language Model (LLM) Infrastructure

The transition to a Zero-Host-CPU architecture unlocks critical capabilities for modern AI pipelines, specifically addressing the GPU memory wall during LLM inference and training.

- **Inference Serving and KV Cache Management:** In state-of-the-art inference serving, the massive context windows required by generative models create severe memory bottlenecks. By utilizing the Monadic Data Plane, inference engines can directly offload the KV cache to NVMe SSDs without host CPU intervention. When integrated with memory managers like PagedAttention [20], GPUDirect Storage enables near-instantaneous page swapping, allowing frameworks (such as vLLM) to maintain massive effective batch sizes without starving the GPU compute cores.
- **Training and Activation Offloading:** During LLM training, intermediate activations and optimizer states dominate GPU HBM. Recent SSD-offloaded training frameworks, such as SSDTrain [51] and TERAIO [52], demonstrate that offloading inactive tensors to PCIe SSDs via GPUDirect Storage can reduce peak memory usage by up to 47% while fully overlapping I/O with computation. By bypassing the CPU bounce buffer, these systems avoid host-memory fragmentation and achieve throughput near the theoretical roofline, effectively expanding the training capacity of resource-constrained nodes [53].
- **The Monadic Sovereign Control Plane:** At the cluster level—particularly when dynamically routing tokens for Mixture of Experts (MoE) architectures—the storage and network data planes become intrinsically linked. By unifying these asynchronous NVMe I/O events and Libfabric network transfers under our C++23 monadic pipeline, we establish a **Monadic Sovereign Control Plane**. This architectural pattern orchestrates GPUDirect RDMA, storage multipathing, and cross-node collective communications entirely in user-space. By handling errors predictively via std::expected rather than relying on kernel-level interrupts, the control plane ensures that the host CPU is completely removed from the AI data path, providing a scalable, highly deterministic foundation for scale-out LLM infrastructure.

6. Intra-Node Scaling: NVLink and Peer-to-Peer Memory Fabrics

While the Libfabric and BlueField-3 data plane addresses the **scale-out** bottleneck of inter-node storage retrieval, modern LLM architectures—particularly those employing Tensor Parallelism (TP) and Mixture of Experts (MoE)—demand extreme **scale-up** bandwidth within the node. In a Zero-Host-CPU architecture, once data is delivered to a primary GPU via GPUDirect Storage, it must be rapidly shared across the local GPU complex without traversing the host PCIe switch.

This is currently achieved via GPUDirect Peer-to-Peer (P2P) [54], operating over NVIDIA's proprietary NVLink memory fabric. While the Monadic Data Plane rigorously champions open-standard transports (such as Libfabric and emerging CXL specifications) to avoid vendor lock-in at the scale-out layer, modern architectures must pragmatically concede to proprietary fabrics to meet the extreme scale-up bandwidth demands of LLM pipelines. With advancements pushing per-GPU

bidirectional bandwidth from 900 GB/s (Hopper NVLink 4) to 1.8 TB/s (Blackwell NVLink 5) [55], the NVLink fabric allows the entire 8-GPU baseboard to act as a single, unified memory domain. Consequently, the monadic control plane can route remote NVMe fetch completions to a single GPU, relying on NVLink's sub-microsecond latency to implicitly serve the remaining GPUs, dramatically reducing redundant network requests and PCIe congestion.

From a software implementation standpoint, the data plane must explicitly opt-in to this topology to prevent silent fallbacks to the host PCIe bus. During the configuration phase, the C++ control plane must invoke cudaDeviceEnablePeerAccess() across all target GPU pairs to authorize direct memory access. Furthermore, when binding Libfabric to the UCX provider, the environment must prioritize the cuda_ipc (Inter-Process Communication) transport (e.g., via UCX_TLS=cuda_copy,cuda_ipc,rc). This ensures that intra-node data exchanges automatically route through the hardware NVLink bridges rather than bouncing through the local loopback network adapter.

7. API Mapping: Control Plane vs. Data Plane

Following the exascale blueprint established by DAOS and the Mercury RPC framework, the Monadic Data Plane partitions Libfabric's API utilization into two distinct operational planes:

- **The Control Plane (Tagged Messaging):** For lightweight, latency-sensitive operations—such as establishing connections, exchanging memory region keys (rkeys), or orchestrating the metadata namespace—the architecture utilizes Libfabric's tagged messaging APIs (fi_tsend and fi_trecv).
- **The Data Plane (Bulk RMA Transfers):** For the massive, zero-copy movement of LLM parameters or KV-caches, the system employs Libfabric's Remote Memory Access (RMA) APIs (fi_readmsg and fi_writemsg).

8. Native GPUDirect RDMA Integration

A foundational requirement of the Monadic Data Plane is moving data directly from the network wire into the GPU without routing through the host CPU's system RAM. Libfabric natively supports this zero-host-CPU paradigm. Rather than manually managing the Linux kernel's nvidia-peermem module or DMA-BUF translation tables, the architecture leverages Libfabric's unified memory registration API. The memory buffer allocated via cudaMalloc (and prepared by DPDK gpudev) is passed directly to the fi_mr_reg() function. Libfabric automatically queries the underlying hardware provider and registers the GPU memory with the network interface. This abstraction allows the SmartNIC to execute one-sided RDMA operations directly against the GPU's PCIe Base Address Register (BAR1), successfully achieving GPUDirect RDMA while completely insulating the developer from the low-level PCIe hardware semantics. By synthesizing Libfabric's universal transport abstraction with DPDK's bare-metal polling and BlueField SNAP's storage virtualization, the dataplane_emu achieves a truly decoupled architecture. The host CPU acts solely as an orchestrator, while the physical data plane operates at wire-speed across any heterogenous cloud environment.

BlueField SNAP and NVMe-over-Fabrics Abstraction

To support the storage component of the emulator, NVIDIA BlueField SNAP (Storage-defined Network Accelerated Processing) is utilized to virtualize NVMe storage over a network fabric. [20, 21] SNAP allows the DPU to emulate a local NVMe controller, presenting networked storage as a local

block device to the host PCIe bus. [20, 22]

SNAP Architecture and Zero-Copy RDMA

The SNAP framework is built upon the Storage Performance Development Kit (SPDK), which provides the underlying block device (bdev) drivers. [21] A critical feature for the Monadic Data Plane is "SNAP-direct," which enables zero-copy DMA between the host (GPU) memory and the remote storage target without intermediate buffering in the DPU's ARM memory. [21, 23] This is enabled by setting the environment variable SNAP_RDMA_ZCOPY_ENABLE=1. [23]

The SNAP service is composed of several infrastructure managers:

- **Emulation Manager:** Handles device discovery and function events. [22]
- **Memory Manager:** Manages SNAP mempools, used only when the zero-copy path is unavailable. [22]
- **IO Manager:** Implements the actual data path for both regular and optimized zero-copy flows. [22]

RPC Configuration and SPDK Integration

Configuration of the SNAP controller and the binding of SPDK bdevs is achieved through JSON-RPC commands. [21, 24] For dataplane_emu, the configuration must be automated to establish the remote NVMe-oF connections during system startup.

The initialization sequence for a SNAP-based NVMe-oF target involves:

1. Subsystem Creation:

Bash

```
nvme_subsystem_create --nqn <NQN>
```

2. Bdev Attachment:

Bash

```
bdev_nvme_attach_controller -b <name> -t rdma -a <IP> -s <port>
```

3. Namespace Creation:

Bash

```
nvme_namespace_create -b <bdev_name> -n 1 --nqn <NQN>
```

4. Controller Creation:

Bash

```
nvme_controller_create --nqn <NQN> --ctrl <id> --pf_id 0
```

This abstraction is essential for hardware-accurate emulation, as it allows the emulator to model the latency characteristics of NVMe-oF while maintaining the simplicity of a local block device interface for the GPU processing kernels.

Unified C++ Architecture and Monadic Design Patterns

The integration of these diverse APIs into dataplane_emu requires a cohesive C++ architectural strategy that prioritizes performance, type safety, and cache efficiency.

Unified Iov and Memory Domain Abstraction

A central challenge in the Monadic Data Plane is managing buffers across different memory domains (CPU, GPU VRAM, and DPU Memory). A unified I/O Vector (MonadicIov) class is required to wrap the metadata needed for direct DMA.

Metadata Field	Type	Description
va	void*	Virtual address in the current process space.
iova	rte_iova_t	I/O virtual address used by the hardware DMA engine.
domain	enum	Specifies if the buffer is CPU, GPU_VRAM, or DPU_MEM.
lkey	uint32_t	Local memory key for RDMA operations.
fd	int	File descriptor for GDS/cuFile operations.

[17, 27, 28]

This abstraction allows the pipeline to remain agnostic of the physical location of the data, provided the appropriate hardware-direct paths have been established during the configuration phase.

Monadic Pipeline with std::expected

To handle the sequence of operations (Receive -> Process -> Store) without traditional error-handling overhead, the architecture utilizes C++23's std::expected<T, E>. This allows for monadic chaining of operations where each step can fail, and errors are propagated through the chain without explicit if checks at every stage. [29, 30]

The "Monadic Bind" pattern (implemented as and_then) allows the developer to compose the data plane as a sequence of functional transformations:

C++

```
auto process_chain = receive_packet(queue)
  .and_then(validate_checksum)
  .and_then(process_gpu_kernel)
  .and_then(store_to_nvme_gds)
  .or_else(handle_failure);
```

[30, 31] In this model, if receive_packet returns a std::unexpected (representing an error like a queue timeout), the subsequent GPU kernel and storage steps are automatically skipped, and the error is handled by the or_else block. [30, 32] This functional approach is not only more readable but also prevents the "pyramid of doom" resulting from deeply nested error checking in high-performance C++ code. [29]

Lock-Free SPSC Ring Buffers and Cache Optimization

Communication between the producer (the NIC/DPU) and the consumer (the GPU kernel) must be wait-free and lock-free to sustain terabit-scale throughput. [33, 34] The Single Producer Single Consumer (SPSC) ring buffer is the ideal primitive for this architecture.

To prevent "false sharing"—a condition where two threads modify different variables on the same cache line, causing constant cache invalidations—the head and tail indices of the ring buffer must be aligned to the cache line size. [34]

C++

```
template <typename T, size_t Capacity>
class SpscRingBuffer {
  alignas(64) std::atomic<size_t> head_{0};
  alignas(64) std::atomic<size_t> tail_{0};

  // Index caching to minimize atomic loads
  size_t head_cached_{0};
  size_t tail_cached_{0};
  T buffer_[Capacity];
};
```

[34] By keeping "cached" copies of the indices, the writer only performs an atomic load of the consumer's index when the cached value indicates the buffer is full, and the reader only loads the producer's index when the buffer appears empty. [34] This optimization has been shown to increase throughput from 5.5 million to 112 million items per second in high-performance networking contexts.

[34]

Memory ordering is equally critical. The producer must use std::memory_order_release when storing the updated tail index to ensure that the data written to the buffer is visible to the consumer. [33, 35] Conversely, the consumer must use std::memory_order_acquire when loading the tail index to establish a "happens-before" relationship with the producer's write. [33, 36, 37]

Deep Dive into libcufile.so and GPUDirect Storage

NVIDIA GPUDirect Storage (GDS) is the key technology for moving data between storage and GPU memory without CPU mediation. [4, 38] The libcufile.so library provides the C API for this direct path.

Core GDS API Signatures

The GDS driver session must be initialized before any I/O operations occur. The following table details the primary functional specifications for the cuFile API.

Function	Signature	Description
Driver Open	cuFileDriverOpen()	Opens the NVFS kernel session and checks platform support. [8, 27]
Handle Reg	cuFileHandleRegister(fh, descr)	Wraps a standard file descriptor for GDS usage. [8]
Buf Reg	cuFileBufRegister(ptr, size, flags)	Pins GPU memory for P2P DMA. [8, 27]
Direct Read	cuFileRead(fh, ptr, size, f_off, b_off)	Executes the direct DMA from storage to VRAM. [8, 27]
Async Read	cuFileReadAsync(fh, ptr, size,..., stream)	Enqueues a read on a specific CUDA stream. [8]

[8, 27]

The cuFileBufRegister operation is a high-latency control-plane task that should be performed during the dataplane_emu initialization phase. Once registered, the memory is "pinned" and "mapped," allowing subsequent cuFileRead calls to be entirely handled by the hardware DMA engine. [7, 8]

Implementation and Fallback Logic

A critical insight for the architect is the handling of DMA-BUF failure. Modern DOCA and GPUNetIO samples use DMA-BUF as the primary method for mapping GPU memory, but they must implement a fallback to nvidia-peermem or host-memory bounce buffers if the kernel does not support the modern direct path. [9, 10]

In a hardware-accurate emulator, these fallbacks must be explicitly modeled. If dataplane_emu detects that the direct path is unavailable, it should either fail (to preserve the "monadic" guarantee) or log the increased latency and CPU utilization caused by the host-memory copy, as this transition changes the performance profile of the system from microsecond-scale to millisecond-scale. [4, 8]

Synthesized C++ Class Blueprint for dataplane_emu

Based on the research, the following class blueprint represents the high-level synthesis of the Monadic Data Plane architecture.

The Device Manager

This component handles the lifecycle of the GPU and NIC/DPU, ensuring that BAR1 is sufficient and drivers are correctly loaded.

C++

```
class DeviceManager {
public:
  // Initialize DPDK, CUDA, and GDS
  std::expected<void, SystemError> initialize() {
    return init_eal()
        .and_then([this] { return init_cuda(); })
        .and_then([this] { return init_cufile(); })
        .and_then([this] { return verify_bar1_size(); });
  }
private:
  std::expected<void, SystemError> verify_bar1_size(); // Checks nvidia-smi props [9]
  std::expected<void, SystemError> init_cufile() {
    if (cuFileDriverOpen() != CU_FILE_SUCCESS)
      return std::unexpected(SystemError::GDS_INIT_FAIL); // [8]
    return {};
  }
};
```

The Monadic Buffer Array

This class manages the unified lovs and handles the DPDK gpudev registration.

C++

```
class MonadicBufferPool {
public:
    MonadicBufferPool(uint16_t gpu_id, size_t pool_size) {
        // Allocate GPU VRAM
        void* ptr = rte_gpu_mem_alloc(gpu_id, pool_size, 4096); // [10, 16]
        // Register as external memory in DPDK
        rte_extmem_register(ptr, pool_size, NULL, iova, 65536); // [17, 18]
        // Map for NIC DMA
        rte_dev_dma_map(nic_id, ptr, iova, pool_size); // [16]
    }
    // Returns a monadic expected value for packet processing
    std::expected<MonadicIov, PoolError> acquire_buffer();
};
```

The Storage Virtualizer (SNAP)

This class wraps the RPC logic for BlueField SNAP to provide an NVMe-oF interface.

C++

```
class SnapStorage {
public:
    std::expected<void, StorageError> connect_remote_target(const std::string& target_nqn) {
        // Construct SPDK RPC command for NVMe-oF attachment
        auto bdev = spdk_rpc_attach_nvme(target_nqn); // [21, 26]
        // Construct SNAP RPC command to create emulated controller
        return snap_rpc_create_controller(bdev)
            .and_then([this] { return set_zero_copy_env(); }); // [23]
    }
private:
    std::expected<void, StorageError> set_zero_copy_env() {
        setenv("SNAP_RDMA_ZCOPY_ENABLE", "1", 1); // [23]
        return {};
    }
};
```

Hardware-Accuracy and Emulation Performance Modeling

A primary objective of dataplane_emu is hardware-accuracy. This requires modeling the nuances of

the PCIe interconnect and the DPU's internal execution units.

Modeling PCIe Topology and Latency

In a hyperscale environment, the physical topology of the PCIe tree significantly impacts performance. If the GPU and the SmartNIC are connected to different CPU sockets, P2P DMA must traverse the QPI or UPI interconnect, which introduces significant latency and jitter. [7] The emulator should utilize lspci -tvvv or nvidia-smi topo -m to build a internal model of the PCIe root complex. [10]

For hardware-accurate latency simulation, the emulator can model the PCIe Gen4/Gen5 bandwidth limits. For instance, a Gen5 x16 link provides a theoretical ~64 GB/s. If the data plane attempts to move data at a rate exceeding this, the emulator must simulate the back-pressure that would occur in a physical NIC's egress buffers. [4, 7]

DPU Offload and ARM Bottlenecks

While the Monadic Data Plane targets zero-host-CPU, it is important to recognize that the DPU's internal ARM cores or the Data Path Accelerator (DPA) RISC-V processors may still be in the control path for certain SNAP operations. [21]

The DPU's performance is often limited by its "Accurate Send Scheduling" and the MTU size of the uplink. [10, 39] For best performance, the emulator should assume a 4096-byte MTU and model the DPU's ability to handle asynchronous I/O commands in parallel across its internal cores. [21, 39]

Conclusion and Strategic Recommendations

The synthesis of GPUDirect, DPDK gpudev, and BlueField SNAP into a Monadic Data Plane represents the apex of current systems engineering for AI and cybersecurity. This architecture effectively disaggregates compute and I/O, allowing the host CPU to focus on high-level orchestration while the data path operates at wire speed.

To successfully build dataplane_emu, the following recommendations are proposed:

1. **Prioritize DMA-BUF:** Given the experimental nature of DMA-BUF on older kernels, the emulator should target Linux Kernel 6.2+ as its baseline to ensure a future-proof memory mapping architecture. [9, 14]
2. **Enforce BAR1 Visibility:** The initialization sequence must include a hard check for BAR1 size. If the GPU is configured with a legacy small BAR (e.g., 256 MiB), the system must warn the user that the zero-copy monadic path is compromised. [8, 9]
3. **Leverage Monadic Chaining:** The use of std::expected and and_then should be mandated across the data plane to ensure that error handling does not become a bottleneck or a source of state corruption in the persistent GPU kernels. [29, 30]
4. **Simulate Interconnect Contention:** To achieve true "hardware-accuracy," the emulator must include a model for PCIe bus contention, particularly in multi-GPU systems where NCCL traffic may compete with storage I/O for the same PCIe switch bandwidth. [2, 7]

The Monadic Data Plane is not merely a performance optimization; it is a fundamental shift toward an autonomous, hardware-accelerated infrastructure. By implementing these research-backed APIs and architectural patterns, dataplane_emu will provide a robust platform for developing the next

generation of hyperscale applications.

References

1. An RDMA-First Object Storage System with SmartNIC Offload - arXiv, https://arxiv.org/html/2509.13997v1
2. Magnum IO Software Stack for Accelerated Data Centers - NVIDIA, https://www.nvidia.com/en-us/data-center/magnum-io/
3. NVIDIA GPUDirect - NVIDIA Developer, https://developer.nvidia.com/gpudirect
4. Quantifying Performance Gains of GPUDirect Storage - IEEE NAS Research, https://www.researchgate.net/publication/365100536_Quantifying_Performance_Gains_of_GPUDirect_Storage
5. Magnum IO Software Developer Kit (SDK), https://developer.nvidia.com/magnum-io
6. PCI BARs and other means of accessing the GPU - envytools git documentation, https://envytools.readthedocs.io/en/latest/hw/bus/bars.html
7. GPUDirect RDMA: The life of a RoCE packet, https://qsysarch.com/posts/gpu-direct-from-rdma-from-pcie-to-using-doca/
8. GDS cuFile API Reference - NVIDIA Documentation, https://docs.nvidia.com/gpudirect-storage/api-reference-guide/index.html
9. GPUNetIO Installation and Setup - NVIDIA Docs, https://docs.nvidia.com/doca/sdk/GPUNetIO-Installation-and-Setup/index.html
10. DOCA GPUNetIO - NVIDIA Docs, https://docs.nvidia.com/doca/archive/2-5-2/doca+gpunetio/index.html
11. Chapter 43. GPUDirect RDMA Peer Memory Client - NVIDIA, https://download.nvidia.com/XFree86/Linux-x86_64/575.57.08/README/nvidia-peermem.html
12. GPUDirect RDMA and GPUDirect Storage — NVIDIA GPU Operator 23.6.2 documentation, https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/23.6.2/gpu-operator-rdma.html
13. GPUDirect RDMA and GPUDirect Storage — NVIDIA GPU Operator, https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-rdma.html
14. Enable CONFIG_PCI_P2PDMA for modern GPUDirect RDMA (DMA-BUF approach) · Issue #12021 · siderolabs/talos - GitHub, https://github.com/siderolabs/talos/issues/12021
15. General-Purpose Graphics Processing Unit (GPU) Library ..., https://doc.dpdk.org/guides/prog_guide/gpudev.html
16. Boosting Inline Packet Processing Using DPDK and GPUdev with ..., https://developer.nvidia.com/blog/optimizing-inline-packet-processing-using-dpdk-and-gpudev-with-gpus/
17. rte_extmem_register(3) — dpdk-doc — Debian testing, https://manpages.debian.org/testing/dpdk-doc/rte_extmem_register.3.en.html
18. DPDK Release 19.02 — Data Plane Development Kit 19.11.0-rc0 documentation, https://dpdk-power-docs.readthedocs.io/en/latest/rel_notes/release_19_02.html
19. NVIDIA DOCA Storage Zero Copy Target RDMA Application Guide, https://docs.nvidia.com/doca/archive/2-9-0/nvidia+doca+storage+zero+copy+target+rdma+application+guide/index.html
20. DOCA SNAP-3 User Guide - NVIDIA Docs,

https://docs.nvidia.com/doca/sdk/DOCA-SNAP-3-User-Guide/index.html
21. NVIDIA BlueField-3 SNAP for NVMe and Virtio-blk v4.5.0, https://docs.nvidia.com/networking/display/nvidia-bluefield-3-snap-for-nvme-and-virtio-blk-v4-5-0.0.pdf
22. NVIDIA BlueField-3 SNAP for NVMe and Virtio-blk v4.4.1, https://docs.nvidia.com/networking/display/nvidia-bluefield-3-snap-for-nvme-and-virtio-blk-v4-4-1.1.pdf
23. Advanced Features - NVIDIA Docs, https://docs.nvidia.com/networking/display/bluefield3snap431/advanced+features
24. SPDK: JSON-RPC - Storage Performance Development Kit, https://spdk.io/doc/jsonrpc.html
25. SNAP RPC Commands - NVIDIA Docs, https://docs.nvidia.com/networking/display/bluefield3snap431/snap+rpc+commands
26. SPDK: NVMe Multipath - Storage Performance Development Kit, https://spdk.io/doc/nvme_multipath.html
27. cuFile API Reference Guide :: CUDA Toolkit Documentation, https://docs.nvidia.com/cuda/archive/11.4.3/cufile-api/index.html
28. DOCA GPUNetIO - NVIDIA Docs - NVIDIA Documentation, https://docs.nvidia.com/doca/archive/2-9-1/doca+gpunetio/index.html
29. std::expected in C++23: A Better Way to Handle Errors | SoftwareCraft, https://softwarecraft.ch/cpp-std-expected/
30. C++23 std::expected — Mastering Monadic Error Handling Pipelines | by Sagar - Medium, https://medium.com/@sagar.necindia/c-23-std-expected-mastering-monadic-error-handling-pipelines-2c62a2eedbaf
31. std::expected - Monadic Extensions - C++ Stories, https://www.cppstories.com/2024/expected-cpp23-monadic/
32. C++23 std::expected Tutorial: Modern Error Handling Guide (2026) | by Sagar - Medium, https://medium.com/@sagar.necindia/cpp-23-std-expected-tutorial-modern-error-handling-guide-2026-9494ebb44e81
33. Circular lock-free buffer - Codemia, https://codemia.io/knowledge-hub/path/circular_lock-free_buffer
34. Optimizing a Ring Buffer for Throughput | Erik Rigtorp, https://rigtorp.se/ringbuffer/
35. Lock-Free Data Engine in C++20 (Part 1/4) | by Josh Brice | Medium, https://medium.com/@joshbrice2025/lock-free-data-engine-in-c-20-part-1-4-e61faf7948bd
36. Lock-Free Ring Buffer - Joe Wood Notes, https://www.joewood.me/posts/lock-free-ring-buffer
37. BlueField-3 DPU Architectural Overview - NVIDIA, https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf
38. UCX (Unified Communication X) Documentation, https://openucx.github.io/ucx/api/v1.10/html/index.html
39. NVIDIA GPUDirect RDMA Technology Overview, https://docs.nvidia.com/cuda/gpudirect-rdma/index.html
40. ofiwg/libfabric: Open Fabric Interfaces - GitHub, https://github.com/ofiwg/libfabric
41. AWS OFI NCCL Plugin - GitHub, https://github.com/aws/aws-ofi-nccl
42. oneAPI Collective Communications Library (oneCCL) - GitHub, https://github.com/uxlfoundation/oneCCL

43. NIXL: NUMA-aware Network Interface - GitHub, https://github.com/ai-dynamo/nixl
44. The Demikernel Datapath OS Architecture for Microsecond-scale Datacenter Systems - SOSP '21, https://www.microsoft.com/en-us/research/publication/the-demikernel-datapath-os-architecture-for-microsecond-scale-datacenter-systems/
45. P2300: std::execution - C++ Sender/Receiver Asynchrony, https://indico.cern.ch/event/1194848/attachments/2524253/4341115/cscs-cern-std-execution-pika.pdf
46. Alibaba Dragonball: Micro-kernel Hypervisor for Cloud Native Workloads - GitHub, https://github.com/kata-containers/kata-containers/tree/main/src/dragonball
47. Learn the Architecture: Memory Systems, Ordering, and Barriers (Armv8-A) - Arm Developer, https://developer.arm.com/documentation/102336/0100/Memory-ordering
48. BypassD: Enabling Fast Userspace Access to Shared SSDs - ASPLOS 2024, https://pages.cs.wisc.edu/~swift/papers/asplos24-bypassd.pdf
49. Oasis: Pooling PCIe Devices Over CXL to Boost Utilization - SOSP 2025, https://joshfried.io/assets/pdf/oasis_sosp25.pdf
50. Design Tradeoffs in CXL-Based Memory Pools for Public Cloud Platforms - Microsoft Research, https://www.microsoft.com/en-us/research/wp-content/uploads/2023/06/2023-CXL-DesignTradeoffs-IEEE-Micro.pdf
51. SSDTrain: An Activation Offloading Framework to SSDs for Faster Large Language Model Training - IEEE Xplore, https://ieeexplore.ieee.org/document/11132754
52. Cost-Efficient LLM Training with Lifetime-Aware Tensor Offloading via GPUDirect Storage (TERAIO) - arXiv, https://arxiv.org/abs/2506.06472
53. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning - ACM, https://dl.acm.org/doi/10.1145/3458817.3476205
54. GPUDirect: Enhancing Data Movement and Access for GPUs - NVIDIA, https://developer.nvidia.com/gpudirect
55. An RDMA-First Object Storage System with SmartNIC Offload - arXiv, https://arxiv.org/html/2509.13997v1

Copyright (c) 2026 SiliconLanguage Foundry. All rights reserved.